

# closure-evidence

## BOB-138 — health check now covers every served port: closure evidence

**Revision: 1** **Last modified:** 2026-08-20T15:02:00Z

### The defect

`docker-compose.yml` service `download-proxy` (container `qbittorrent-proxy`) serves BOTH 7186 and 7187 from a single process, but its health check probed only 7186. Measured 2026-08-20 while the container reported Up 4 hours (healthy):

```
curl --max-time 6 http://localhost:7186/ -> HTTP 200 in 0.096s
curl --max-time 6 http://localhost:7187/ -> HTTP 000 after 6.004s
```

The merge search service — the product’s primary capability — had been dead for roughly two hours and no signal in the stack could observe it. §11.4.201: the guard asserted a PROXY signal (one port answers) instead of the real condition.

### The fix

The health check now probes both ports. `curl` is ABSENT from the `python:3.12-alpine` image (verified in-container), so the `python -c urllib` probe is deliberate; `urlopen` raises on non-2xx or timeout, so either port failing exits non-zero.

## Polarity evidence — BOTH directions

### (§11.4.201(1))

A guard must refuse the broken state AND accept the healthy one.  
Golden-bad is port 7199, verified unbound (`ss -ltn` shows no listener),  
executed inside the real container:

| # | command          | state            | exit | meaning                             |
|---|------------------|------------------|------|-------------------------------------|
| A | new two-probe    | both ports live  | 0    | accepts a genuinely healthy service |
| B | new two-probe    | second port dead | 1    | the second probe is load-bearing    |
| C | old single-probe | second port dead | 0    | the false pass — i.e. the bug       |

Row C is the defect reproduced; row B is the same state refused after the fix; row A proves the fix does not refuse legitimate traffic.

### A note on an INVALID earlier attempt (§11.4.6)

A first polarity run was executed against the live 7187 wedge, intending to use it as the golden-bad. Between the forensics and that run the wedge SELF-CLEARED (7187 returned to HTTP 200 in 0.021s), so both the old and the new command exited 0 — not because they behave alike, but because nothing was broken. That run proved nothing and is recorded here as void rather than quietly dropped. The table above is the re-run against a port that is genuinely dead.

## Static guard

`scripts/pre_build/check_cm_healthcheck_covers_served_ports.sh` asserts every port declared in `config/served_ports.yaml` appears in that service's health check.

- RED (pre-fix): FAIL — `download-proxy`: serves [7186, 7187] but its healthcheck probes none of [7187], with the other services passing (so it is not a gate that simply refuses everything).
- GREEN (post-fix): PASS (5 services verified).

The gate also refuses to report success when it checked zero services, because a quiet zero from a blind instrument is indistinguishable from a clean tree (§11.4.201(6)).

## A false positive the gate caught in its own input data

The manifest first declared `qbittorrent-proxy-go`: serves [7186, 7187, 7188], taken from `CLAUDE.md` prose. That is wrong: `qBittorrent-go/Dockerfile:16` runs a single binary (`CMD ["/app/qbittorrent-proxy"]`) which binds one port (`ServerPort = MERGE_SERVICE_PORT = 7187`). The gate consequently failed a service whose health check was already correct — the §11.4.201(1) false refusal this gate exists to prevent, committed in its own DATA. The manifest was corrected against source rather than prose, and the entry now carries that provenance.

## Residual (tracked, not silently skipped)

- The gate is boba-local. Its logic carries no boba literal and belongs in `constitution/scripts/gates/` per §11.4.177, but a concurrent agent is editing that directory and a two-writer race there risks losing work. Upstreaming is tracked, not skipped (§11.4.197).
- `CLAUDE.md` states the Go profile “replaces the Python proxy on 7186, 7187, 7188”, which the Dockerfile contradicts. Documentation defect, filed separately.

## Live verification — and a restart-level trap worth recording

The fix was first applied with `./start.sh --reload-python`. The container came back healthy and both ports answered, which looked like success. The cache-bust check CLAUDE.md mandates (“VERIFY served content matches committed code”) showed otherwise:

```
$ podman inspect qbittorrent-proxy --format
'{{json .Config.Healthcheck.Test}}'
["CMD-SHELL","python -c \"import urllib.request;
urllib.request.urlopen(
    'http://localhost:7186/', timeout=5)\" || exit 1"]      <-
STILL THE OLD CHECK
```

`--reload-python` only RESTARTS the container, so a `docker-compose.yml` change is not applied. CLAUDE.md states this explicitly: a compose change requires `./start.sh --recreate`. Without the served-vs-committed check, this would have been reported as fixed while the old single-probe check was still running — the exact §11.4.108 SOURCE-vs-RUNTIME gap the guard exists to catch.

After `./start.sh --recreate`:

```
$ podman inspect qbittorrent-proxy --format
'{{json .Config.Healthcheck.Test}}'
["CMD-SHELL","python -c \"import urllib.request as u;
    u.urlopen('http://localhost:7186/', timeout=5);
    u.urlopen('http://localhost:7187/health', timeout=5)\" || exit
1"]

$ podman ps --filter name=qbittorrent-proxy --format '{{.Status}}'
Up 40 seconds (healthy)

$ curl --max-time 8 localhost:7186/      -> HTTP 200
$ curl --max-time 8 localhost:7187/health -> HTTP 200
```

The two-probe check is now the one actually running, and the container reaches (healthy) through it — so the health signal now depends on the merge service being alive.

## **Also observed during the restart (feeds BOB-137)**

```
level=warning msg="StopSignal SIGTERM failed to stop container  
qbittorrent-proxy  
in 10 seconds, resorting to SIGKILL"
```

The service does not shut down cleanly on SIGTERM. That is consistent with the BOB-137 blocked-thread evidence and is recorded there as an additional symptom.